

Architecture

Decisions, trade offs and production considerations arising from the ledger core in this repository. Where a claim is about the code, the file is named.

1. Append only at scale

What breaks first is the balance query, and it breaks quadratically

`Ledger.balanceMinor` has no cache and no running total: it filters the account's whole history and sums it. Every balance in the system comes from that one function.

The cost is not the sum but who calls it, and how often. `assessOverdraftFees` calls it once per day across the whole window, at every day close:

```
days_to_reassess x entries_in_account_history
```

In production the window is not six days but every day the account has been open, since a backdated entry can reach any of them. So both terms grow linearly in account age, and **the day close is quadratic in it**. Ten events over six days hides that completely; at 100 times the volume it is 10,000 times the work, in a batch window that must finish before the business opens. Interest capitalization has the same shape, and so does a projection rebuild, the operation append only is supposed to make cheap.

Storage is not the problem. Storage is the cheapest part of an append only design, and the one people worry about first.

Where the design accumulates unbounded state

The restatement window. Nothing limits how far back a `valueDate` may reach, so no day is ever finished. This causes

Problem	How the snapshot bounds it
Balance query cost	Scan from the snapshot, not from inception
Fee reassessment loop	Only days in the open period can move, so only those are reassessed
Interest recomputation	Accruals in a sealed period are final, and are read rather than recalculated
Projection rebuild time	Rebuild from the last snapshot, not from the first event

It needs no change to the append only log. The log stays the source of truth and the snapshot stays a cache, so a snapshot can be discarded and rebuilt whenever it is doubted. That is what makes it cheap: it cannot corrupt anything, so it can run alongside the existing path before anything depends on it. The second change, once the first is in, is a TTL on the hold register.

2. Value dated entries in a UAE licensed bank

The operational surface

A back valued entry does not change one number. It changes every number derived from the days it reaches, and most of those have already been sent somewhere.

Statements already issued become wrong. The bank chooses between reissuing, which invites the question of what else changed, and an adjustment line in the current period, which reads worse but leaves the audit trail intact.

Fees already charged may no longer be due, or may now be due. The fee engine reassesses closed days for this reason, so a customer can be charged today for a day that closed a week ago, and the system cannot yet generate that explanation.

the quadratic cost above and most of section 2.

The hold register. `HoldRegister` has no expiry, so an authorization never settled and never voided holds funds forever. This replay hides it: its only unsettled authorization was declined.

The log and the ledger. Unbounded by design, and correctly so. These are history. The problem is not that they grow but that everything is recomputed from them.

Derived state. Every closing balance is recalculated on demand. The property that makes a balance always correct and never stale is the one that makes it expensive.

The cheapest structural change

A periodic balance snapshot, plus a sealed period flag. One table and one job.

```
snapshot(account_id, value_date)
-> closing_balance, as_of_sequence
```

A nightly job writes a closing balance per account for the day just sealed. `balanceMinor` reads the nearest snapshot and applies only later entries, bounding the scan by entries since the snapshot rather than by the whole history. The sealed flag forbids a value date inside a closed period; anything reaching further posts to the open period as a prior period adjustment.

It is the highest leverage change available because one mechanism bounds four problems:

One property is worth naming. No rule here reads two accounts, so the log shards cleanly on `account_id` with no coordination, and `test/order-independence.e2e-spec.ts` demonstrates the property that makes that safe. It stops being true the moment double entry arrives, because a balanced transaction spans accounts. Worth knowing before choosing the sharding key.

Interest already credited is recomputed, silently, because interest is derived. Silent is fine inside a window nobody has reported. Across a reported period it is a restatement.

Downstream consumers have already consumed. Monitoring, credit decisioning, limit management and the warehouse feed all read the old numbers, and each has its own idea of what a correction means.

The regulatory surface

Seven surfaces a value date can move a transaction across. Each is named by the exposure rather than by the instrument governing it, deliberately: instruments are amended and superseded, the shapes are not, and quoting circular numbers

from memory asserts what the writer cannot check. Every one needs a compliance opinion before go live. None of this is one.

Consumer protection. A fee charged retroactively is a fee disclosure and an error correction question at once. A fee a later reversal shows to have been the bank's own error sits further in again, and this implementation cannot tell the two apart, which is the subject of its one failing test.

AML and CFT. Monitoring is date sensitive. Back valuing moves a transaction into or out of a window, and can change whether a set of transactions constitutes structuring. Reporting clocks run from detection, so an alert raised today concerns activity dated weeks ago.

Regulatory reporting. Returns are period based. A value date landing in a submitted period changes a filed figure, which is a resubmission and a conversation rather than a routine correction.

Accounting. Interest recognition attaches to a period. An entry crossing a reporting boundary is a prior period adjustment if material, and materiality is assessed in aggregate, so the exposure is the total volume of back valuing rather than any single entry.

VAT. An explicit fee is a taxable supply with a tax point, unlike a margin based product. Back dating or reversing one moves that tax point, potentially across a filed return.

Dormancy. Dormancy keys off the date of last customer activity. A back valued entry changes that date retroactively, moving an account into or out of dormancy and, at the far end, unclaimed balance transfer.

Shari'ah compliance. This model is conventional. Interest on a credit balance and a flat overdraft fee do not transfer to an Islamic product, where the equivalents are profit distribution

and a cost recovery charge on a different basis. A bank running both needs a product aware accrual engine.

The one control I would add before going live

A closed period lock, with back valuing beyond it requiring dual authorization and a reason code.

Value dates may not fall inside a sealed accounting period. Inside the open period, back valuing beyond T plus two business days requires maker checker approval, a reason from a closed list, and an immutable record of who approved it. Anything reaching further posts as a prior period adjustment, never as a silent rewrite of a closed period.

I would choose this over tamper evident hash chaining or a full bitemporal audit trail, for three reasons.

It bounds how far a correction can reach, rather than recording where it reached. Every problem in this section follows from a correction reaching something already reported. A lock stops it reaching; an audit trail tells you afterwards, which is useful and not the same thing.

It supplies the missing field. The reason code is the concept whose absence produces the one failing test. Once a reversal says why, the system can distinguish a bank error, where consumer protection rules require the fees back, from a legitimate customer return, where the account really was overdrawn. Today it applies one rule to both and is wrong about one of them every time.

It is the same mechanism as the scaling fix. The sealed period flag from section 1 and this lock are one feature. Getting the performance work and the regulatory control from one change is unusual and worth taking.

Runners up, in order: a TTL on the hold register, hash chaining the log, and a customer facing adjustment advice generated whenever a restatement crosses a statement boundary.

3. Authorization lifecycle

What this model supports

An authorization in `authorization.types.ts` has three states: approved, settled, declined. Reading the paths in `event-handlers.ts` rather than the state list, there are **three** ways one ends other than a settlement for the amount held, and a fourth where it never ends at all.

1. Declined at creation. The hold would take available balance below zero, so no hold is created and the state is terminal. *Scenario:* the ordinary one, a customer without the funds, or with funds already committed to an earlier hold. *Mandated, and implemented:* record the decline with the balances that produced it. It is an output an operator needs, not an error to swallow, and storing it lets a later settlement tell an authorization that was refused from one never requested.

2. Refused as a duplicate identifier. A second authorization under an identifier already in the register never becomes one. *Scenario:* a retry. The acquirer did not receive the response, or a queue redelivered the message. *Mandated, and implemented:* refuse, and leave the existing authorization untouched, since treating the retry as new creates a second hold for one purchase and locks twice the money. Note this is refusal, not idempotency: an idempotent path would return the original outcome so the retry succeeds.

3. Settled for an amount other than the amount held. The authorization terminates and the whole hold is released,

whatever the difference. *Scenario:* routine, and what happens in this replay. A restaurant bill before a tip, a fuel pre authorization, a basket that changed before capture. *Mandated:* the implemented behaviour is only conditionally right. Releasing the whole hold suits a single presentment product and is wrong for a multi presentment one, where a hotel folio or split shipment presents again and the residual should stay held. Set the policy per product, and treat a global choice as a defect whichever way it is set.

4. It never ends. An approved authorization never presented holds funds indefinitely. No expiry, no acquirer void, no sweep. *Scenario:* the merchant abandons the sale, the terminal drops after approval, or a pre authorization is never captured. *Mandated:* this path should not exist, so the mandate is the expiry below. The gap is invisible here because the only unsettled authorization was declined.

One guard keeps a terminated authorization terminated: a settlement against an already settled authorization is refused rather than posted, so a duplicate presentment cannot debit twice.

What production must mandate

Four endings this model has no path for, plus one case that is their mirror.

Expiry. The merchant never presents. Mandate a TTL per product, not one global value: card present retail around seven days, hotel and vehicle rental thirty, fuel at a pump much

shorter. On expiry, append a release event and free the hold, never mutating the authorization. A settlement after expiry is then a force post rather than a revived dead hold.

Acquirer initiated void or reversal. The merchant cancels at the terminal, or a final authorization replaces an estimated one. Mandate immediate release, full or partial, on receipt, and no further settlement against that identifier. The cheapest hold to release, and the one customers notice most when it is not, because they are standing at the counter.

Over settlement beyond tolerance. Tips, fuel dispensed after the pre authorization and currency movement all exceed the hold legitimately. Mandate a tolerance per merchant category, commonly fifteen to twenty percent for restaurants and fuel, zero for most others. Within tolerance, post and release. Outside it, post anyway, since the issuer is generally obliged to honour the presentment, and raise an exception so the excess is investigated rather than absorbed.

Settlement with no authorization. Not an ending, since there is nothing to end, but the mirror of every ending above, and

the one path this model has and gets wrong. Offline and floor limit transactions, chip fallback, stand in processing during an outage, late presentment. This implementation refuses these because the brief requires it, and no real issuer could. Mandate a suspense account posting with an exception raised, then chargeback rights if the authorization is genuinely absent. Never silently decline: the acquirer is unpaid and the customer has the goods.

Account state change between authorization and settlement. A freeze, a sanctions match, a court order, a closure, or a death. Mandate that the existing hold survives, since those funds are already committed, while new authorizations are declined. A settlement against a frozen account still posts, with the freeze enforced at the balance level rather than by dropping the posting, so the bank creates no unreconciled difference with the scheme. Escalate to compliance rather than resolving in the ledger.

One more sits outside the lifecycle but ends the economics: a **chargeback** after settlement, which reverses the posting under scheme timelines entirely separate from anything here.

4. What was cut, and what each cut defers

Ordered by the size of the risk, not the size of the change.

The cut	What is absent	The risk it defers
Double entry	Contra accounts for fee income, interest expense, settlement clearing and suspense	No trial balance, and no way to prove value is conserved across the book rather than within one account. Finance cannot see fee income or interest expense at all. A ledger that cannot be balanced is a record, not a ledger, and this is the largest cut here
Closed periods and a back value limit	Any bound on how far back a value date may reach	Everything in section 2. Every statement and every regulatory return stays permanently provisional
Snapshots	Any stored closing balance; every one is recomputed	The quadratic day close from section 1, and a projection rebuild that goes from seconds to hours
Idempotency	A key on an event, and any deduplication	A retried delivery double posts. In a system fed by a queue with at least once delivery this is routine, not an edge case
Concurrency control	Locking and optimistic versioning. The replay is single threaded	Two authorizations arriving together read the same available balance and both approve, and the account goes overdrawn past the limit meant to prevent exactly that
Authorization expiry	A TTL, an acquirer void, and a sweep	Holds leak, and customer funds are locked with no event that will ever free them
Reversal reason codes	Any field saying why a reversal happened	The consumer protection exposure in section 2, and the one failing test
Tamper evidence	Hash chaining and signing. The log is append only by construction, with frozen records	An append only log with no tamper evidence is not an audit record, whatever the code does. Cheaper to add before there is history to migrate
A business calendar	Weekends, UAE public holidays, timezone, cutoff	Accruals and value dates diverge from the business calendar, and the divergence compounds. The UAE weekend moved to Saturday and Sunday for the federal sector in January 2022, so the rule is not even a constant
A day count convention	Actual/365, Actual/360, 30/360. A flat rate per day instead	The accrual will not agree with product terms or with treasury, and the difference is small per day and systematic
Debit interest	Any overdraft pricing beyond a flat fee	An overdraft is mispriced, and regressively, since a small overdraft pays the same fee as a large one
A multi currency fee schedule	A fee in any currency but AED	An account in another currency cannot be charged, which the code raises on rather than guessing. Honest, and still a gap
Access control and segregation of duties	Authentication, authorization, maker checker	Every control in section 2 assumes somebody can be prevented from doing something, and nothing here can prevent anything
Observability	Metrics, structured logs, a reconciliation report	The first production incident is diagnosed by reading source code
Event schema versioning	A version on an event	The first change to an event shape makes the historical log unreadable by the current code, which is the failure that turns an append only log from an asset into a liability
Persistence, and everything downstream	Storage, transactions, recovery. Out of scope by instruction rather than by choice	Every guarantee in this document is a guarantee about one process's memory. A restart loses the book, and nothing here has been shown to survive one